Communication Patterns: Synchronous vs Asynchronous

What is communication pattern and its types?

Communication pattern is the way microservices talk to each other to share information or perform a task. Since each service is independent, they need clear ways to communicate. Choosing the right pattern helps the system be fast, reliable, and easy to maintain.

Types of Communication Patterns

1. Synchronous Communication

- Services talk directly and wait for a response.
- One service call another, waits for it to finish, and then continues.
- This is simple but can slow down the system if a service is busy or down.

Example: In a hotel booking system, the Booking Service calls the Room Service to check availability. It waits for Room Service to respond before confirming the booking.

2. Asynchronous Communication

- Services send messages and do not wait for an immediate response.
- The receiving service processes the message when it can.
- This makes the system more resilient and scalable.

Example: In a banking system, when a customer transfers money:

- The Transfer Service publishes a MoneyTransferRequested event.
- Notification Service listens to this event and sends an SMS.
- Transfer Service does not wait for the SMS to be sent.

How do you achieve Synchronous Communication in Microservices?

HTTP/REST Calls

- Services use REST APIs over HTTP to talk to each other.
- One service sends a request, waits for the response, and continues.

GraphQL

- GraphQL allows a service to provide a single endpoint where other services or clients can ask for only the data they need.
- When a service makes a GraphQL request, it waits for the response before moving forward, just like a normal synchronous call.
- This is especially useful when we need to get information from multiple services at once, because GraphQL can combine the data in a single request instead of making many separate REST calls.

gRPC (Remote Procedure Call)

- A faster alternative to REST, especially for internal service-to-service communication.
- Services call methods on other services like normal functions.

SOAP or other RPC protocols

- Used in legacy systems for strict contracts between services.

Refer : API Design Decision - Rest vs GraphQL

What is gRPC and how it is useful?

gRPC stands for google Remote Procedure Call.

It is a communication framework developed by Google that allows services to talk to each other efficiently. Instead of sending text-based messages like REST, gRPC uses binary data with Protocol Buffers, which makes communication faster, lighter and suitable for microservices and real-time systems.

gRPC is a way for services to communicate with each other quickly and efficiently. It allows one service to call a method on another service as if it were a local function, even if the services are on different servers.

gRPC is faster than traditional REST because it uses a binary protocol instead of text, and it supports streaming data in both directions. It is especially useful for internal communication between microservices in large systems. gRPC supports different ways for services to communicate. These patterns make it flexible for different types of tasks.

1. Unary RPC

- What it is: One service sends a single request and gets a single response.
- Example: In a smart city system, Emergency Service asks Traffic Service for the current traffic status at a specific location. Traffic Service replies with one response.

2. Server Streaming RPC

- What it is: One request is sent, but the server sends multiple responses over time.
- Example: Traffic Service sends live traffic updates to Public Display Service. The display gets updates continuously without sending new requests.

3. Client Streaming RPC

- What it is: The client sends multiple requests and waits for a single response from the server.

  Example: Sensors in a building send temperature readings every few seconds to Building Management Service.

- Once all readings are sent, the service calculates and responds with the average temperature.

4. Bidirectional Streaming RPC

- What it is: Both client and server send messages continuously.
- Example: In a hospital monitoring system, Patient Monitoring Service streams live vital signs to Doctor Service. Doctor Service can send alerts or instructions back in real time.

Real-World Example:

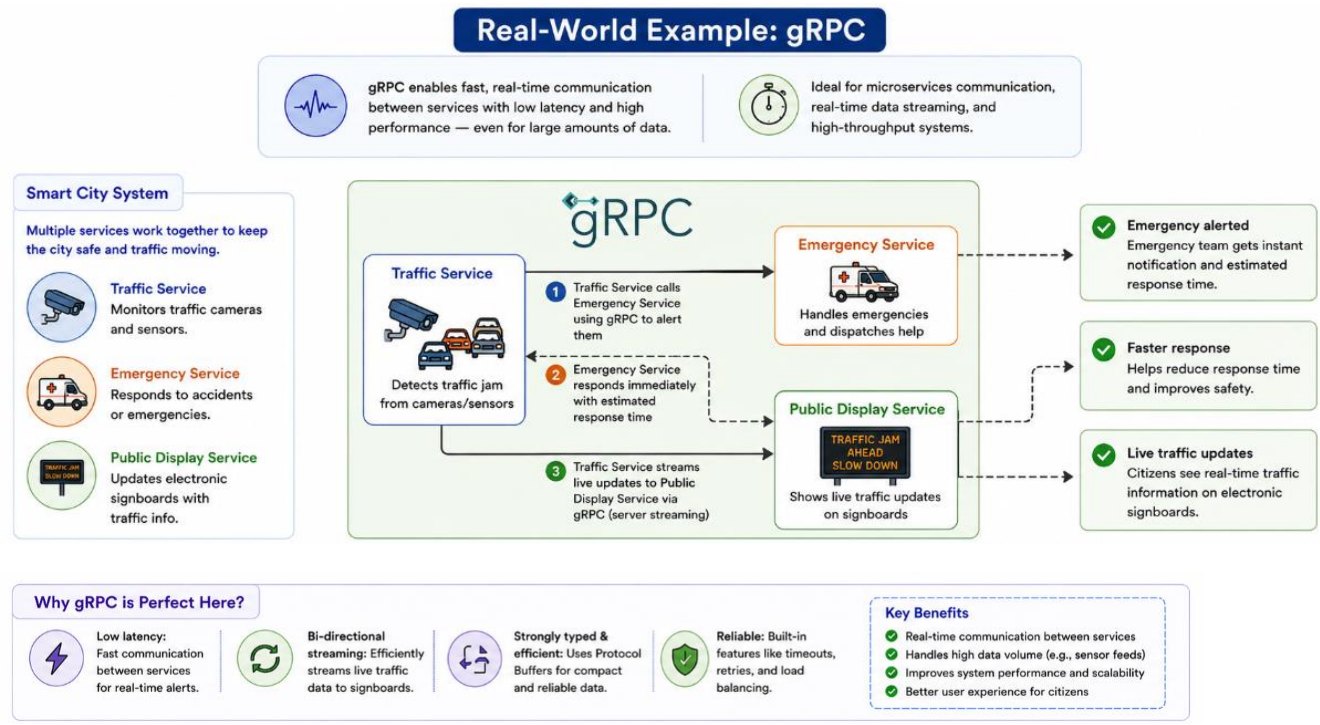
Imagine a smart city system with multiple services:

- Traffic Service: monitors traffic cameras and sensors.
- Emergency Service: responds to accidents or emergencies.
- Public Display Service: updates electronic signboards with traffic info.

Flow using gRPC:

1. Traffic Service detects a traffic jam and calls Emergency Service using gRPC to alert them.
2. Emergency Service responds immediately with the estimated response time.
3. Traffic Service streams live updates to Public Display Service to show current traffic on signboards.

Here, gRPC allows fast, real-time communication between services, even with large amounts of data like live sensor feeds.



What is asynchronous communication and how it is useful?

Asynchronous communication is a way for services to talk without waiting for a response immediately. One service sends a message or event, and the other processes it when it can. This makes systems more scalable, reliable, and efficient.

How It Works

1. Service A sends a message to Service B.
2. Service A continues its work without waiting for Service B to respond.
3. Service B processes the message later and may trigger another event.

Real-World Example:

Imagine a university notification system: When a student finishes the admission process in the Admission Service, a message is sent out saying "**New Student Admitted**"

This message is not sent to one single service. Instead, it goes to a message broker (like Kafka or RabbitMQ.

1. The Library Service listens and creates a library account for the student.
2. The Hostel Service listens and checks for room availability.
3. The ID Card Service listens and starts preparing the ID card.

The Admission Service does not wait for any of them to finish. It just sends the message and continues admitting other students. Each service does its work when it receives the event. This pattern makes the system more flexible. If tomorrow a new service is added, like a Sports Club Service, it can also listen to the same event without touching the Admission Service.

Commonly used tools for asynchronous communication in microservices:

1. Apache Kafka
2. RabbitMQ
3. Amazon SQS (Simple Queue Service)
4. Google Pub/Sub

How can you implement Asynchronous communication in Microservice?

We can use a message broker to handle asynchronous messages. Kafka and RabbitMQ are two common choices.

Example using Spring with RabbitMQ.

1. Add message sender in Admission Service

```
@Service
public class AdmissionService {
    private final RabbitTemplate rabbitTemplate;

    public AdmissionService(RabbitTemplate rabbitTemplate) {
        this.rabbitTemplate = rabbitTemplate;
    }

    public void admitStudent(String studentId) {
        // business logic
        rabbitTemplate.convertAndSend("studentExchange", "student.admitted", studentId);
    }
}
```

2. Add message listener in Library Service

```
@Service
public class LibraryListener {

    @RabbitListener(queues = "studentQueue")
    public void handleNewAdmission(String studentId) {
        // create library account
        System.out.println("Library account created for: " + studentId);
    }
}
```

3. Add simple configuration

```
@Configuration
public class RabbitConfig {

    @Bean
    public TopicExchange exchange() {
        return new TopicExchange("studentExchange");
    }

    @Bean
    public Queue queue() {
        return new Queue("studentQueue");
    }

    @Bean
    public Binding binding(Queue queue, TopicExchange exchange) {
        return BindingBuilder.bind(queue).to(exchange).with("student.admitted");
    }
}
```

How this works:

1. Admission Service sends an event.
2. Library Service listens to the event and works on it when ready.
3. Both services are independent.
4. If the Library Service is down, Admission Service still works.
5. When the Library Service comes back, it will receive the pending messages.

When to use Asynchronous and when to use synchronous communication?

Synchronous Communication Synchronous means, one service sends a request to another and waits until it gets the response. Only after getting the response does it continue its work.

When to Use It

• When we need an instant response.

• When the request and response are part of one workflow.

• When real-time confirmation matters.

Example: In an online shopping portal, when we place an order, the Order Service needs an immediate confirmation from the Payment Service before marking the order as complete. This is a synchronous call because the Order Service must wait for the Payment Service to confirm the payment before moving ahead.

Asynchronous Communication

Asynchronous means, a service sends a message and continues its work without waiting for the other service to respond immediately. The other service processes the message later and sends the result when ready.

When to Use It

• When the task can happen later without blocking the main flow.

• When we want services to be independent and loosely connected.

• When performance and scalability are more important than real-time replies.

Example: In the same shopping portal, after an order is confirmed, a Notification Service sends an email or SMS to the user. The Order Service need not to wait for the Notification Service to finish sending messages. It just sends a message to a queue like Kafka or RabbitMQ and continues processing.

How does Apache Kafka works and why it is fast?

Think of Kafka as a large, really fast post office that receives messages (data) from various sources and sends them to their respective destinations.

Let's imagine a system like **Uber**, but we'll focus only on how Kafka fits in, not the full product.

Real world example: Ride booking system.

Think about how many events happen when a ride is booked. • A rider books a cab.

• A driver gets the ride request.

• The price is calculated.

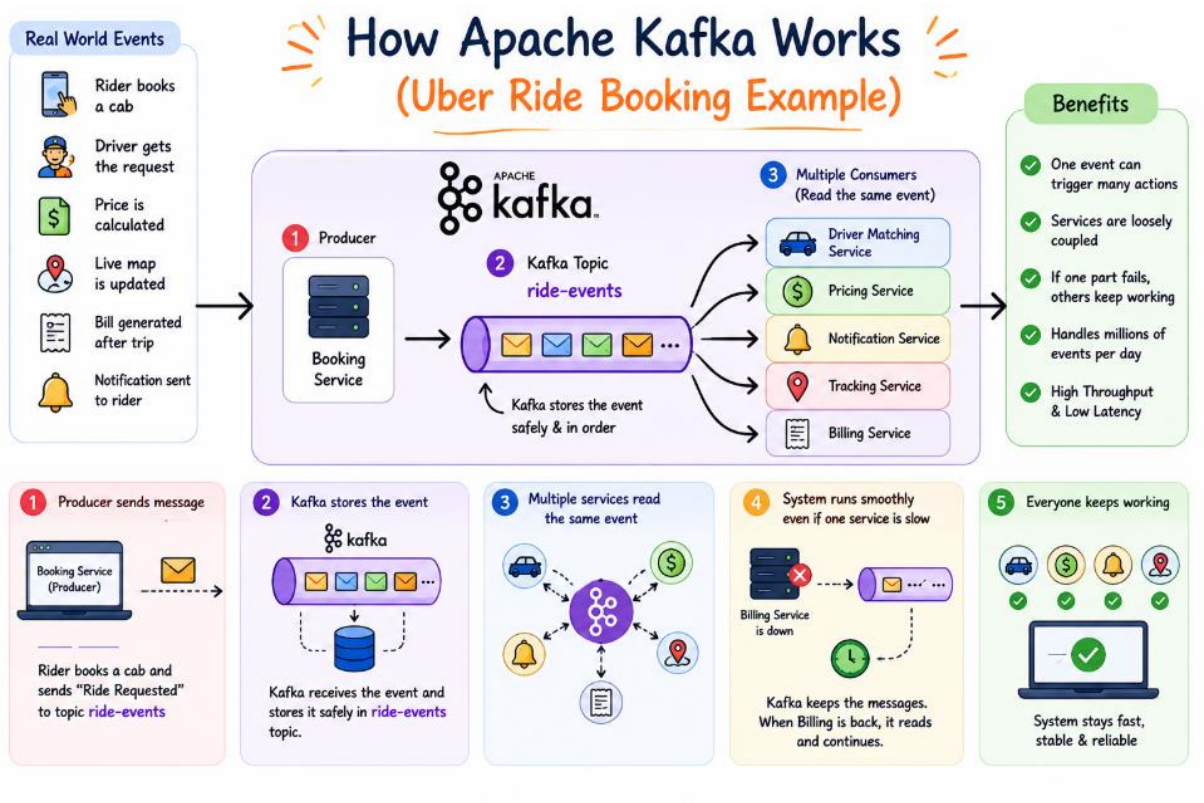
• A live map is updated.

• A bill is generated after the trip ends.

• A notification is sent to the rider.

If all these services talked directly to each other, it would get messy.

If one service fails, the others might break too. This is where Kafka becomes powerful. Here's how it works step by step.



Step 1: Producer sends message When a rider books a cab, the Booking Service acts as a producer. It sends a message like "Ride Requested" to a Kafka topic called ride-events.

Step 2: Kafka stores the event Kafka receives the event and keeps it in the ride-events topic. It doesn't care which service will use it. It just stores the message safely and in order.

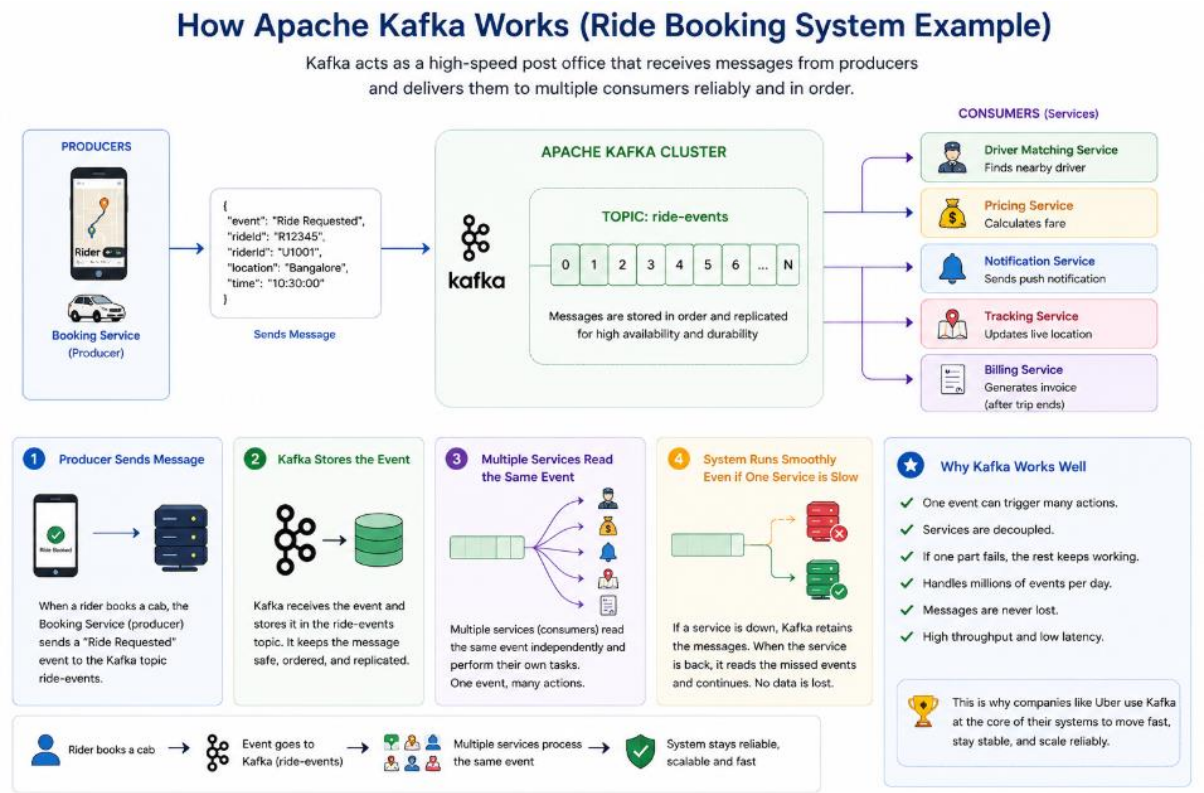
Step 3: Multiple services read the same event

1. Driver Matching Service reads the event and finds a nearby driver.
2. Pricing Service reads the same event and calculates the fare.
3. Notification Service reads it to send a push message to the rider.
4. Tracking Service reads it to update the live location.
5. Billing Service reads it after the ride ends to create the invoice

Step 4: System runs smoothly even if one service is slow Let's say the Billing Service is down for a few minutes. No problem. Kafka still keeps the messages. When Billing comes back, it reads the stored events and continues processing. The rest of the system keeps running.

Why Kafka works well here

- One event can trigger many actions.
- Services are not tightly connected.
- If one part fails, the rest of the system keeps working.
- The system can handle millions of ride events per day without crashing.



This is exactly why companies like Uber use Kafka at the center of their system. It lets the platform move fast, stay stable, and scale without losing messages.

Why is Apache Kafka so fast?

Can you tell some real-world use cases for Kafka?

Real-World Usage of Apache Kafka

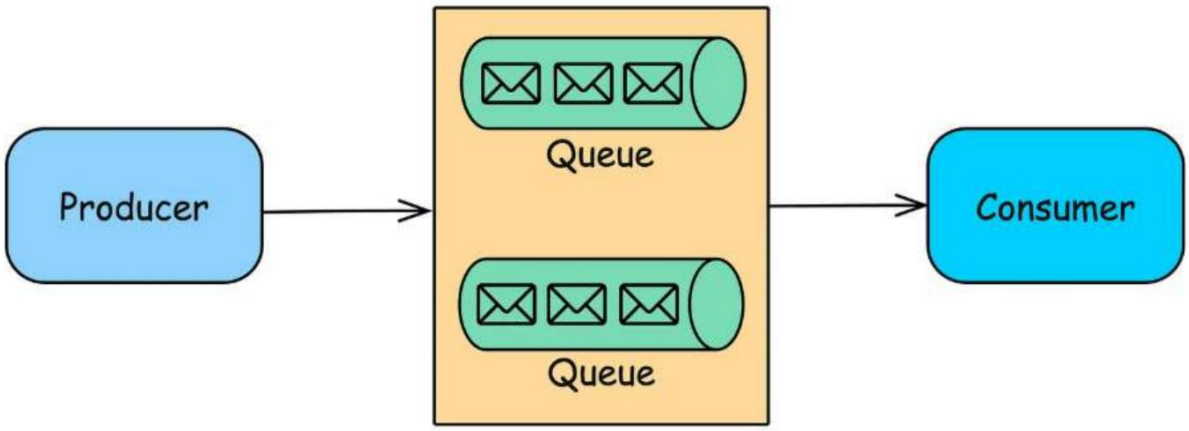
- **Netflix** uses Kafka to apply a recommendation in real-time while you're watching TV shows.

- **Uber** uses Kafka to gather taxi user and trip data in real time and they will use it to compute and forecast demand and then they can compute the infamous search pricing in real-time to know how much to charge you for a ride in case there is a lot of high demand.
- **LinkedIn** uses Kafka to prevent spam and collect user interactions to make better connection recommendations in real time.

What are message queues?

A message queue is a system that allows services to send messages to each other without waiting.

Messages are stored in a queue until the receiving service is ready to process them.



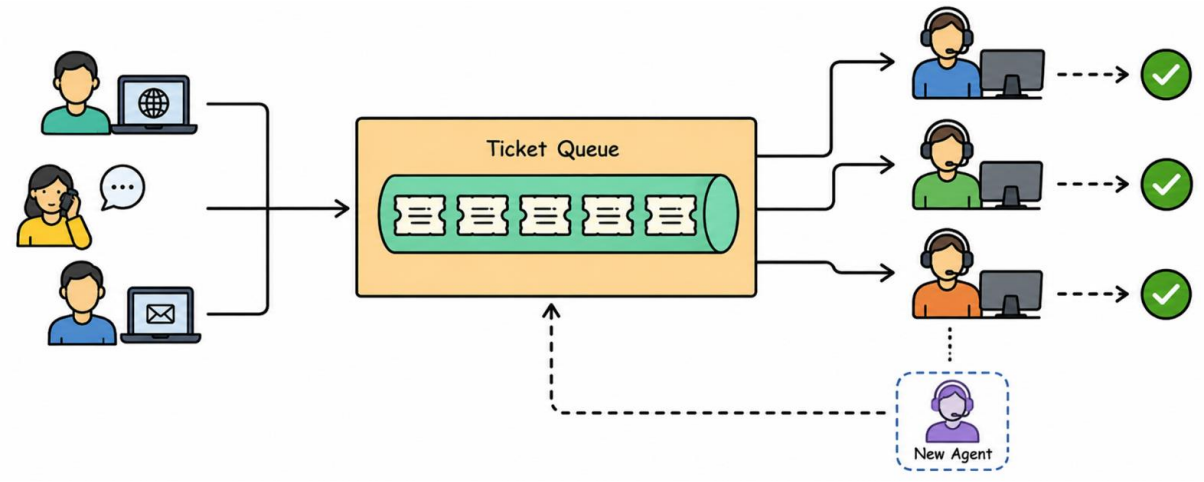
Real world example: Call Centre Support System

1. Imagine a call centre with multiple support agents:
2. Customers submit support tickets online or via phone.
3. The ticket system receives all tickets and puts them into a queue.
4. Agent 1 picks a ticket and starts resolving it.
5. Agent 2 picks another ticket and works on it.
6. If a new agent joins the team, they can also start picking tickets from the same queue without disrupting the system.
7. Each ticket waits in the queue until an agent is ready to process it.
8. If an agent is busy or temporarily unavailable, the ticket stays safely in the queue. Nothing is lost, and all tickets are handled efficiently.

How Message Queues Work:

1. **Producer**: The service that sends the message.
    a. Customers submitting tickets act as the producer.
2. **Queue**: The storage place where messages wait.
    a. The ticket system is the queue.

    b. Tickets wait here until an agent is available.
3. **Consumer**: The service that reads and processes the message.
    a. The support agents are consumers.
    b. Each agent picks a ticket from the queue and resolves it.
4. **Broker/Queue Manager**: The software or service that manages the message queue, handles the delivery of messages, and ensures that messages are routed correctly between producers and consumers.
5. **Acknowledgment**: The consumer tells the queue the message is processed.
    a. Once agent finishes resolving ticket, the message is removed from the queue.



Examples

1. AWS SQS: A managed service for message queuing.
2. RabbitMQ & Kafka: Popular message brokers for various architectures.
3. IBM MQ: Enterprise-grade message queuing middleware.

Types of Message Queues: There are several types of message queues, each designed to solve specific problems:

1. Point-to-Point (P2P) Queue:

2. Publish/Subscribe (Pub/Sub) Queue

3. Priority Queue

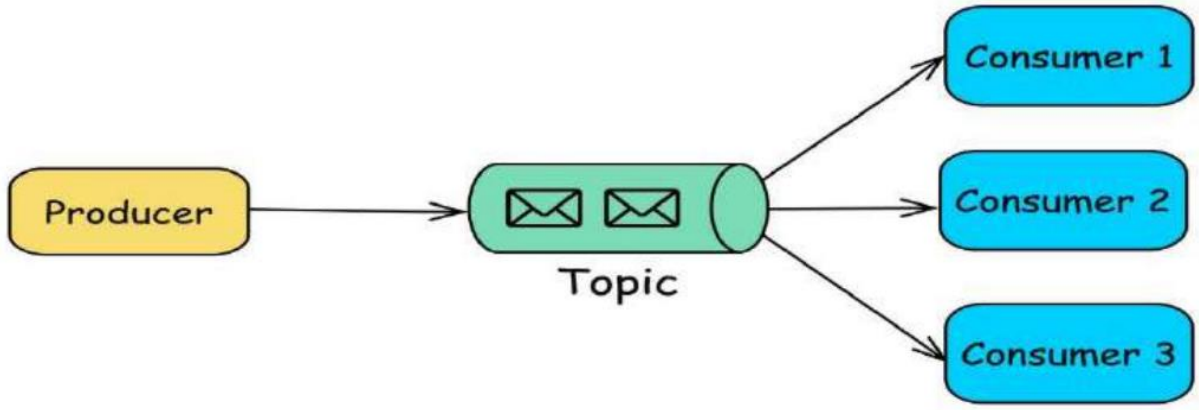
4. Dead Letter Queue (DLQ)

1. Point-to-Point (P2P) Queue:

- In this model, messages are sent from one producer to one consumer.
- Used when a message needs to be processed by a single consumer, such as in task processing systems.

2. Publish/Subscribe (Pub/Sub) Queue:

In this model, messages are published to a topic, and multiple consumers can subscribe to that topic to receive messages. Used for broadcasting messages to multiple consumers, such as in notification systems.

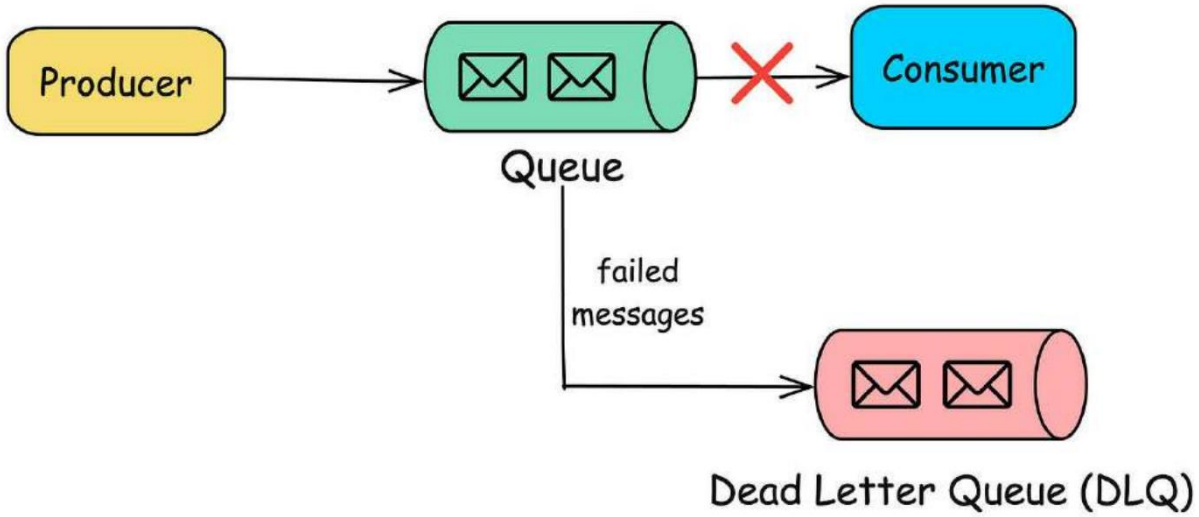


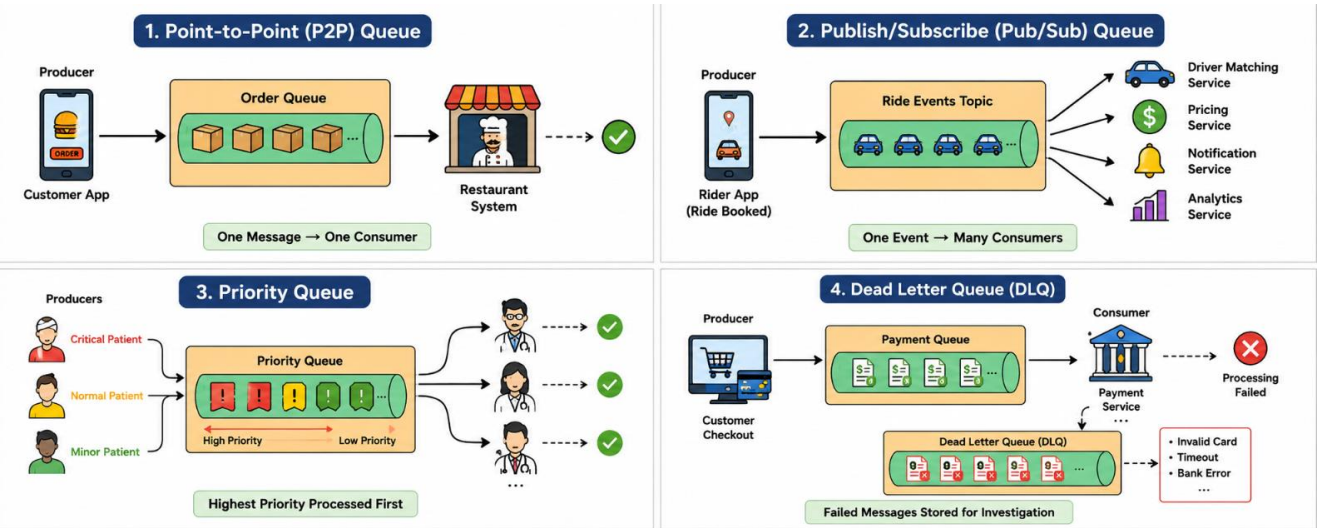
3. Priority Queue:

- Messages in the queue are assigned priorities, and higher-priority messages are processed before lower priority ones.
- Used when certain tasks need to be handled more urgently than others.

4. Dead Letter Queue (DLQ):

A special type of queue where messages that cannot be processed (due to errors or retries) are sent. Useful for troubleshooting and handling failed messages.

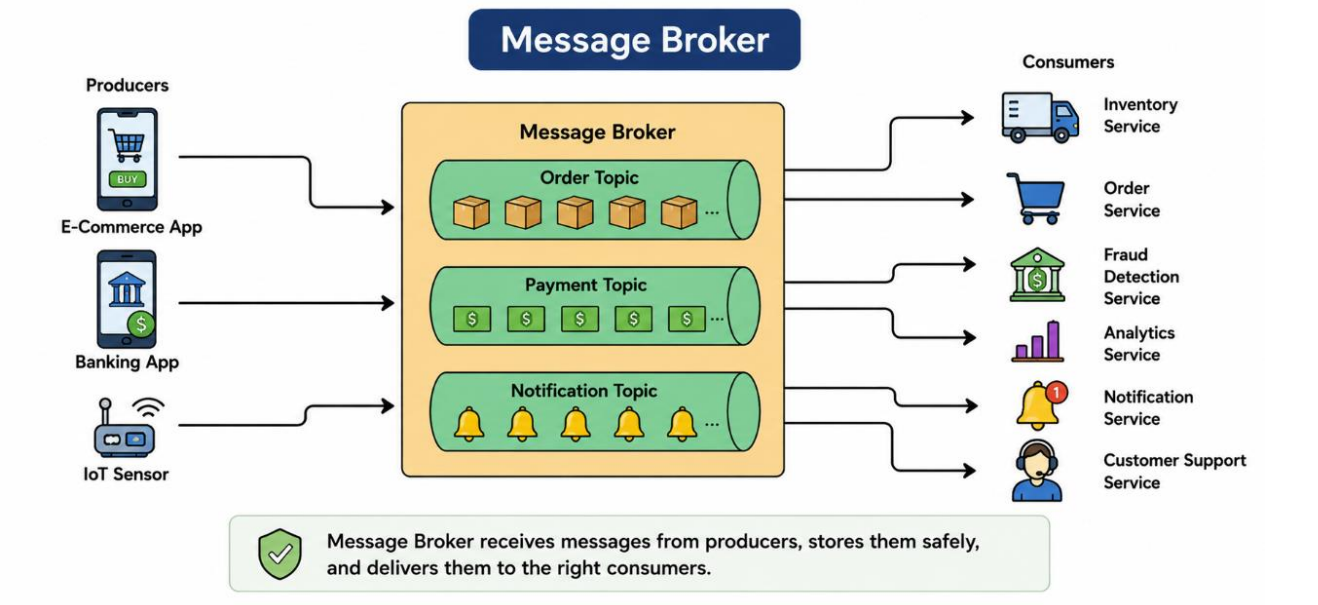




What is message broker?

A message broker is a software that enables applications, systems, and services to communicate with each other and exchange information.

Think of it like **post office** where it receives messages from one service and delivers them to other services. The broker ensures messages are delivered safely, in order, and without losing any, even if some services are busy or temporarily down.



Message brokers are very useful in microservices because they decouple services. Producers (services that send messages) don't need to know who the consumers are or when they will process the message.

Popular Message Brokers in the Market

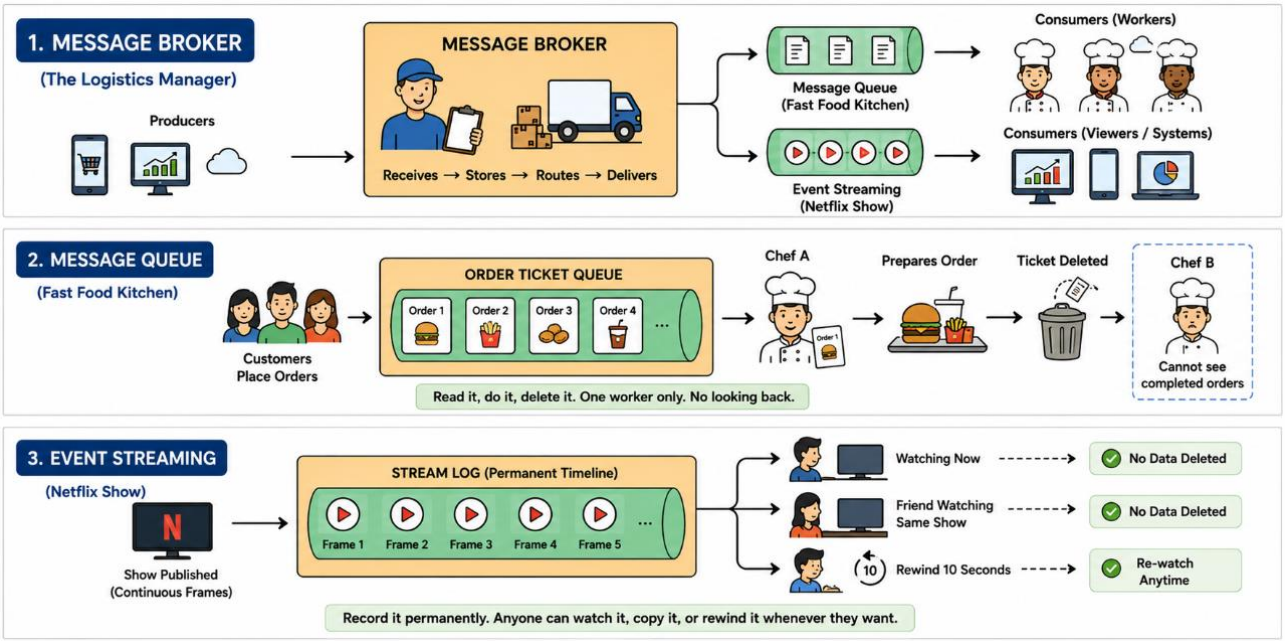
1. RabbitMQ: Easy to use, reliable, works well for most queue-based tasks. RabbitMQ is an opensource message broker that implements the Advanced Message Queuing Protocol (AMQP). It helps in decoupling services and ensures reliable message delivery.

2. Apache Kafka: Handles very high volumes of messages, good for streaming and event-driven systems.

3. ActiveMQ: Another general-purpose messaging system, supports multiple protocols.

4. Amazon SQS: Cloud-based, fully managed queue service, no server setup required.

5. Google Pub/Sub: Cloud messaging service, scales automatically, good for event-driven systems.



When to use message queues?

Message queues are useful whenever services need to communicate without waiting for each other. They help systems work smoothly even if some parts are slow or temporarily down, and make scaling easier.

1. Microservices Architecture

- Problem: Microservices need to communicate with each other, but direct communication can lead to tight coupling and cascading failures.
- Solution: Use message queues to enable asynchronous communication between microservices, allowing each service to operate independently and resiliently.

2. Task Scheduling and Background Processing

- Problem: Certain tasks, such as image processing or sending emails, are time-consuming and should not block the main application flow.
- Solution: Offload these tasks to a message queue and have background workers (consumers) process them asynchronously.

3. Event-Driven Architectures

- Problem: Events need to be propagated to multiple services or components, but direct communication would be inefficient.
- Solution: Use a Pub/Sub message queue to broadcast events to all interested consumers, ensuring that all parts of the system receive the necessary updates.

4. Load Leveling

- Problem: Sudden spikes in requests can overwhelm a system, leading to degraded performance or failures.
- Solution: Queue incoming requests using a message queue and process them at a steady rate, ensuring that the system remains stable under load.

5. Reliable Communication

- Problem: Communication between components needs to be reliable, even in the face of network or service failures.
- Solution: Use persistent message queues to ensure that messages are not lost and can be retried if delivery fails.

Real time Examples:

• Instagram (Meta): Used RabbitMQ in early systems to queue tasks like photo processing before scaling up to Kafka.

• Reddit: Uses RabbitMQ for background jobs like email delivery and notification processing.
• Spotify: Uses Kafka to handle streaming events, like logging song plays and updating recommendations.

• Uber: Uses Kafka to manage events from drivers, riders, and trip updates in real-time.

• Airbnb: Uses RabbitMQ to queue background tasks such as sending booking confirmations and notifications.

• LinkedIn: Uses Kafka to process activity feeds, notifications, and data pipelines efficiently

When to use Kafka, RabbitMQ and Pub/sub?

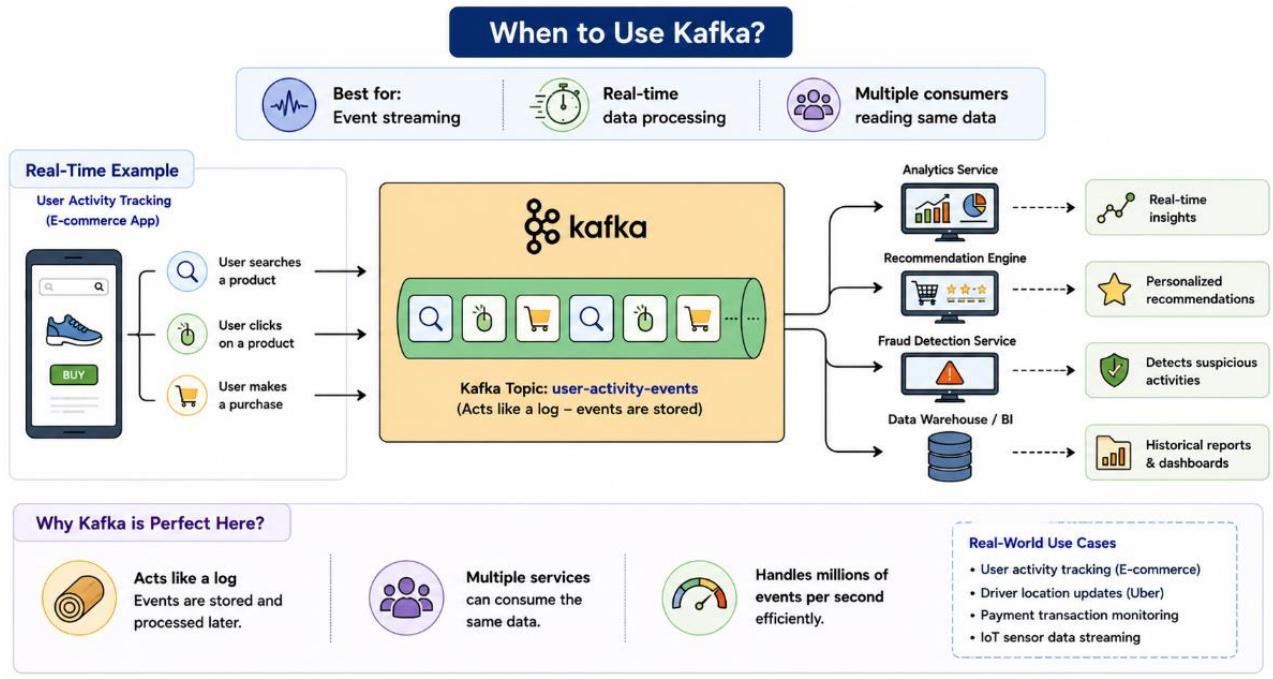
When to use Kafka?

Best for: Event streaming, Real-time data processing, Multiple consumers reading same data

Real-Time Example:

User Activity Tracking (E-commerce App) or Driver location updates in Uber

- Imagine Amazon wants to track user actions (searches, clicks, purchases).
- This data needs to be analysed in real-time for recommendations, analytics.
- Kafka is perfect here because:
  - It acts like a log, where events are stored and processed later.
  - Multiple services (analytics, recommendation engine.) can consume the same data.
  - It can handle millions of events per second efficiently.



When to use RabbitMQ?

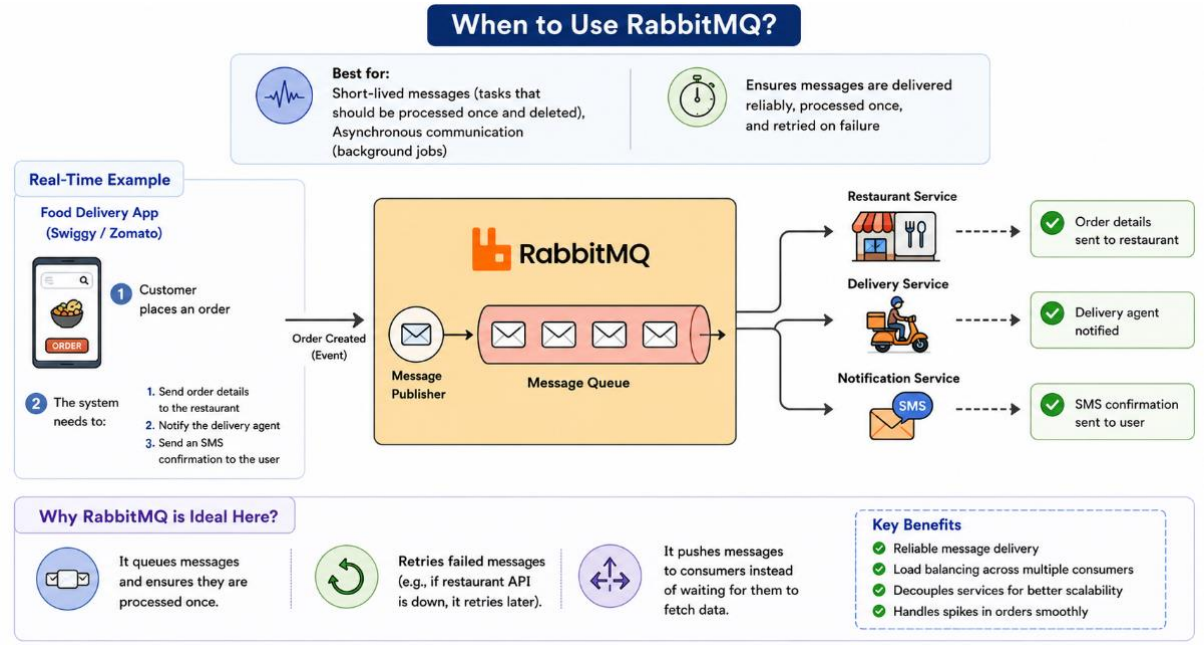
Best for: Short-lived messages (tasks that should be processed once and deleted), Asynchronous communication (background jobs).

Real-Time Example: Food Delivery App (Swiggy/Zomato)

- A customer places an order.
- The system needs to:
  1. Send order details to the restaurant.
  2. Notify the delivery agent.
  3. Send an SMS confirmation to the user.

RabbitMQ is ideal here because:

1. It queues messages and ensures they are processed once.
2. Retries failed messages (e.g., if restaurant API is down, it retries later).
3. It pushes messages to consumers instead of waiting for them to fetch data.

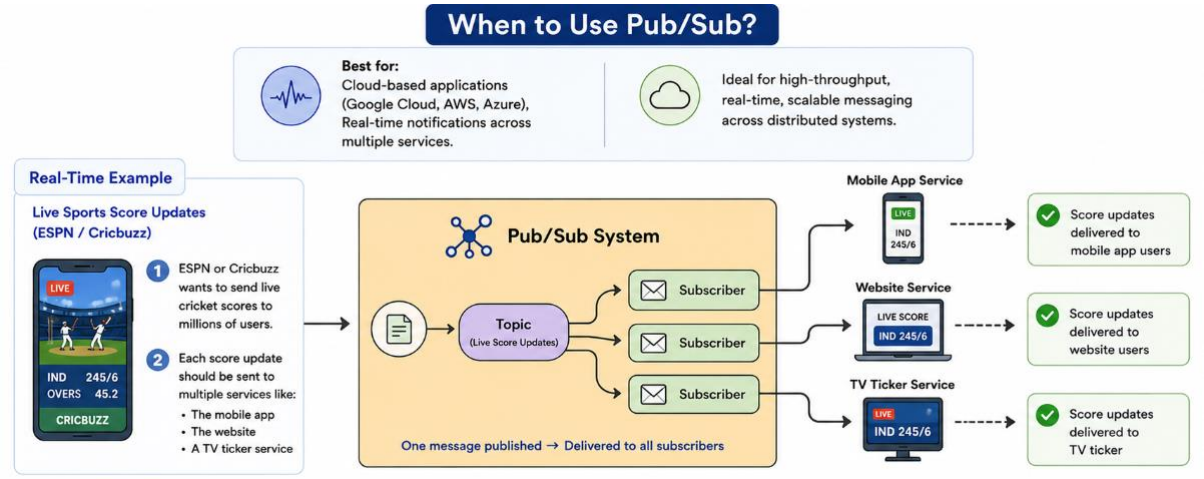


When to use Pub/Sub?

Best for: Cloud-based applications (Google Cloud, AWS, Azure), Real-time notifications across multiple services.

Real-Time Example: Live Sports Score Updates

- ESPN or Cric buzz wants to send live cricket scores to millions of users.
- Each score update should be sent to multiple services like:
  - The mobile app
  - The website
  - A TV ticker service



Difference between Kafka and RabbitMQ?

Feature	Kafka	RabbitMQ
Type	Distributed Event Streaming Platform	Traditional Message Broker / Message Queue
Use Cases	Real-time analytics, log aggregation, streaming pipelines, event-driven microservices	Task queues, background jobs, asynchronous workflows, reliable message delivery
Message Model	Publish–Subscribe with Consumer Groups	Queue-based (Pub/Sub + Work Queues + Routing)
Message Ordering	Guaranteed within a partition	Not guaranteed across multiple consumers
Delivery Guarantees	At-most-once, At-least-once, Exactly-once (with Kafka Streams)	At-most-once, At-least-once using acknowledgements and retries
Replay Capability	✅ Yes — Consumers can replay historical events anytime	❌ No — Messages are removed after consumption unless explicitly stored
Retention	Time-based or size-based retention	Messages removed after acknowledgement
Storage	Events are persisted on disk for a configurable period	Primarily designed for message delivery, not long-term storage
Consumer Behaviour	Consumers pull messages from Kafka	Broker pushes messages to consumers
High Throughput	Designed for extremely high throughput (millions of events/sec)	Lower throughput compared to Kafka
Scalability	Horizontal scaling through partitions and brokers	Scales well but not optimized for massive event streaming workloads
Latency	Very low latency for streaming workloads	Very low latency for task processing workloads
Best For	Streaming large volumes of ordered events	Reliable task processing and message delivery
When to Use	Data pipelines, log processing, event streaming, analytics platforms	Job queues, notifications, order processing, workflow orchestration
Real-World Examples	Netflix activity streams, LinkedIn event tracking, Uber ride events, IoT telemetry	Order processing, email sending, SMS notifications, payment processing workflows